

```
#include <iostream> using namespace std;
```

```
int mergeAndCount(int arr[], int temp[], int left, int mid, int right) {
    int i = left;
    int j = mid;
    int k = left;
    int invCount = 0;

    while (i <= mid - 1 && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
            invCount += (mid - i);
        }
    }

    while (i <= mid - 1) {
        temp[k++] = arr[i++];
    }

    while (j <= right) {
        temp[k++] = arr[j++];
    }

    for (i = left; i <= right; i++) {
        arr[i] = temp[i];
    }

    return invCount; }

int mergeSortAndCount(int arr[], int temp[], int left, int right) {
```

```
    int invCount = 0;
    if (left < right) {
        int mid = (left + right) / 2;
        invCount += mergeSortAndCount(arr, temp, left, mid);
        invCount += mergeSortAndCount(arr, temp, mid + 1, right);
        invCount += mergeAndCount(arr, temp, left, mid + 1, right);
    }
    return invCount; }
```

```
int countInversions(int arr[], int n) {
    int temp[n];
    return mergeSortAndCount(arr, temp, 0, n - 1); }
```

```
int main() {
    int arr[] = { 1, 20, 6, 4, 5 };
    int n = sizeof(arr) / sizeof(arr[0]);
```

```
    cout << "Original array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
```

```
    int inversions = countInversions(arr, n);
```

```
    cout << "Number of inversions: " << inversions << endl;
```

```
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
```

```
    return 0; }
```